Limited Time Offer: **Up to 25% off select mock interviews** 🎉

Patterns

Multi-step Processes

Learn about multi-step processes and how to handle them in your system design with sagas, workflow systems, and durable execution.

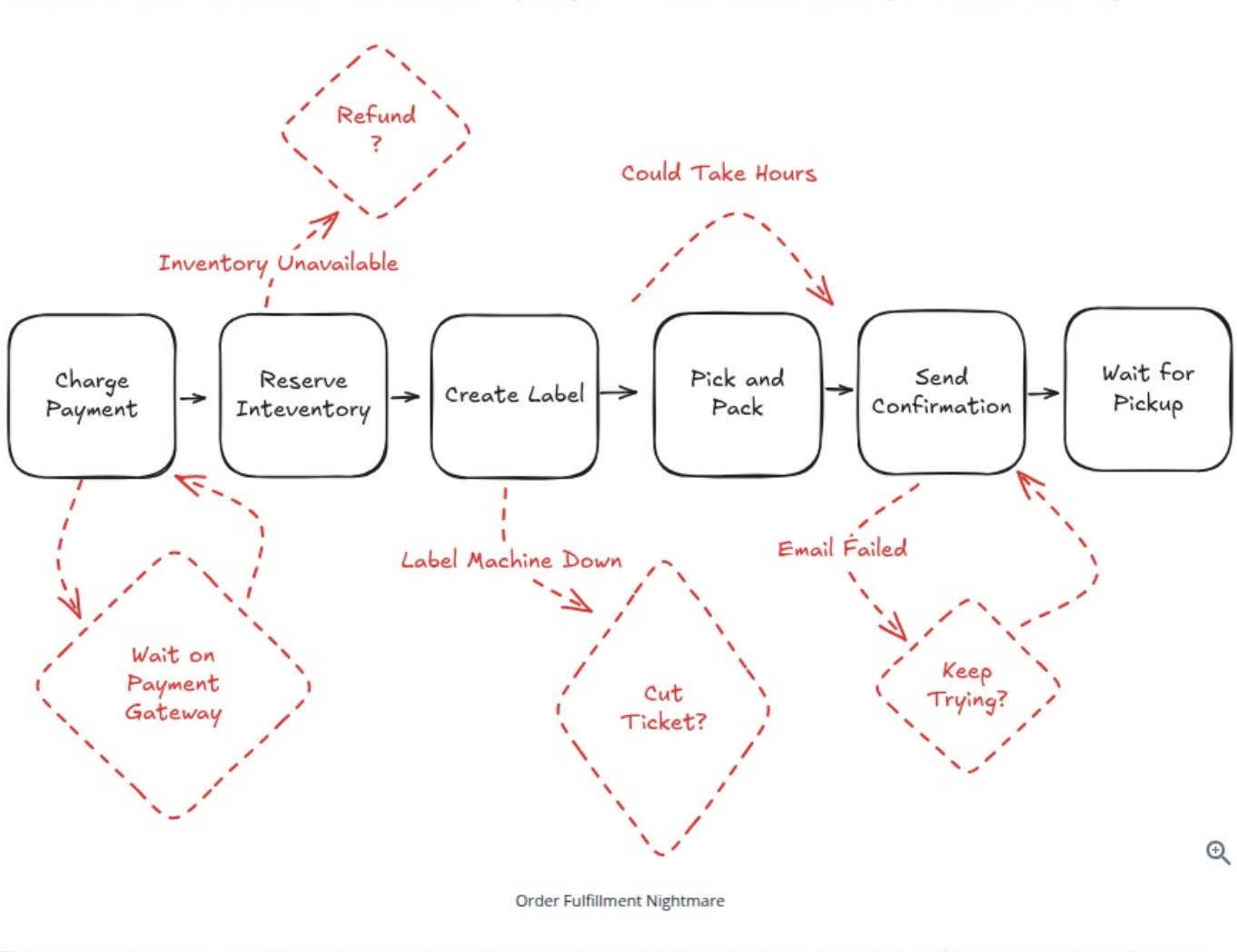
Multi-step Processes

⚙️ Real production systems must survive failures, retries, and long-running operations spanning hours or days. Often they take the form of multi-step processes or sagas which involve the coordination of multiple services and systems. This is a continual source of operational and design challenges for engineers, with a variety of different solutions.

The Challenge

Many real-world systems end up coordinating dozens or even hundreds of different services and systems in order to complete a user's request, but building reliable multi-step processes in distributed systems is startlingly hard. While clean systems like databases often get to deal with a single "write" or "read", real applications often need to talk to dozens of (flaky) services to do the user's bidding, and doing this quickly and reliably is a common challenge. Jimmy Bogard has a great talk about this titled **"Six Little Lines of Fail"** with the premise that distributed systems make even a simple sequence of steps like this surprisingly hard (if you haven't had to deal with systems like this before, it's a great watch).

Consider an e-commerce order fulfillment workflow: charge payment, reserve inventory, create shipping label, wait for a human to pick up the item, send confirmation email, and wait for pickup. Each step involves calling different services or waiting on humans, any of which might fail or timeout. Some steps require us to call out to external systems (like a payment gateway) and wait for them to complete. During the orchestration, your server might crash or be deployed to. And maybe you want to make a change to the ordering or nature of steps! The messy complexity of business needs and real-world infrastructure quickly breaks down our otherwise pure flow chart of steps.



Order Fulfillment Nightmare

There are, of course, patches we can organically make to processes like this. We can fortify each service to handle failures, add compensating actions (like refunds if we can't find the inventory) in every place, use delay queues and hooks to handle waits and human tasks, but overall each of these things makes the system more complex and brittle. We interweave system-level concerns (crashes, retries, failures) with business-level concerns (what happens if we can't find the item?). Not a great design!

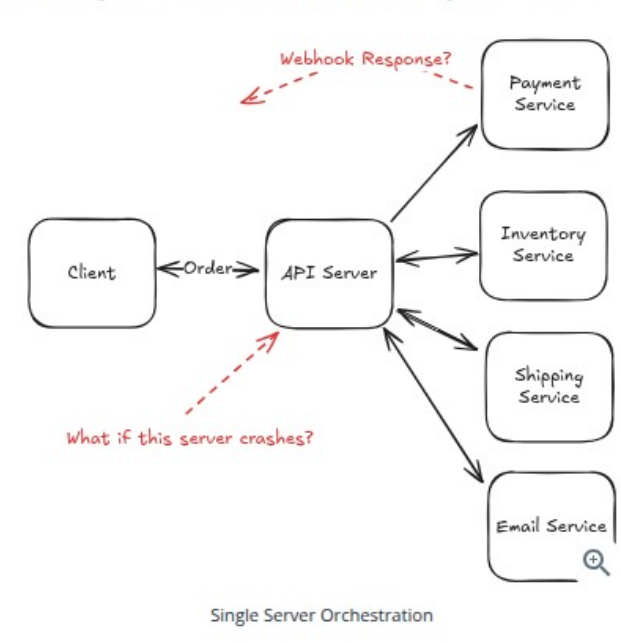
Workflow systems and durable execution are the solutions to this problem, and they show up in many system design interviews, particularly when there is a lot of state and a lot of failure handling. Interviewers love to ask questions about this because it dominates the oncall rotation for many production teams and gets at the heart of what makes many distributed systems hard to build well. In this article, we'll cover what they are, how they work, and how to use them in your interviews.

Solutions

Let's work through different approaches to building reliable multi-step processes, starting simple and building up to sophisticated workflow systems.

Single Server Orchestration

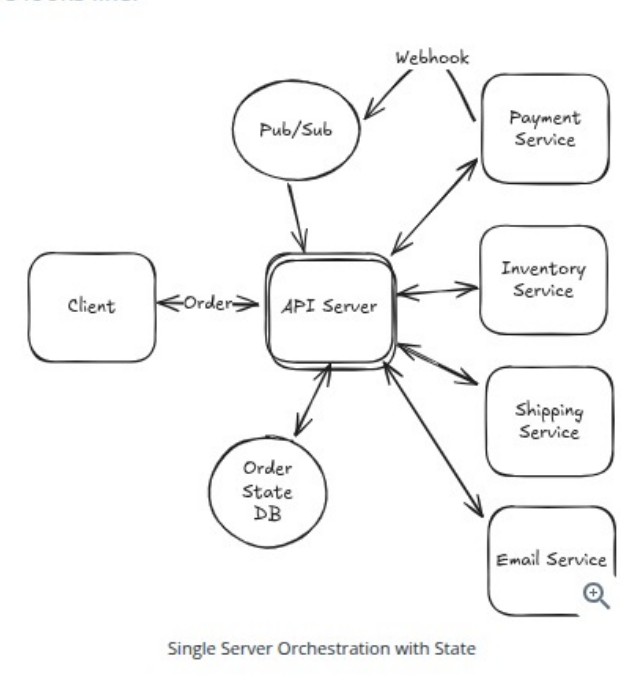
The most straightforward solution is the one most engineers start with: it's to orchestrate everything from a single server, often in a single service call. Your API server receives the order request, calls each service sequentially, and returns the result. If you didn't know any better, this would be where you'd start!



Single Server Orchestration

Not all problems involve complex state management or failure handling. And for these simple cases, single-server orchestration is a perfectly fine solution. But it has serious problems as soon as you need reliability guarantees and more complex coordination. What happens when the server crashes after charging payment but before reserving inventory? When the server restarts, it has no memory of what happened. Or how can we ensure the webhook callback we get from our payment gateway makes its way back to the initiating API server? Are we stuck with a single host with no way to scale or replicate?

You might try to solve this by adding state persistence between each step and maybe a pub/sub system to route callbacks. Now your architecture looks like:



Single Server Orchestration with State

We'll solve the callback with pub/sub. And we can scale out our API servers now because when they start up, they can read their state from the database. But this quickly becomes complex, and we've created more problems than we've solved. As an example:

- You're manually building a state machine with careful database checkpoints after each step. What if you have multiple API servers? Who picks up the dropped work?
- You still haven't solved *compensation* (how do we respond to failures?). What if inventory reservation fails? You need to refund the payment. What if shipping fails? You need to release the inventory reservation.

The architecture becomes a tangled mess of state management, error handling, and compensation logic scattered across your application servers.

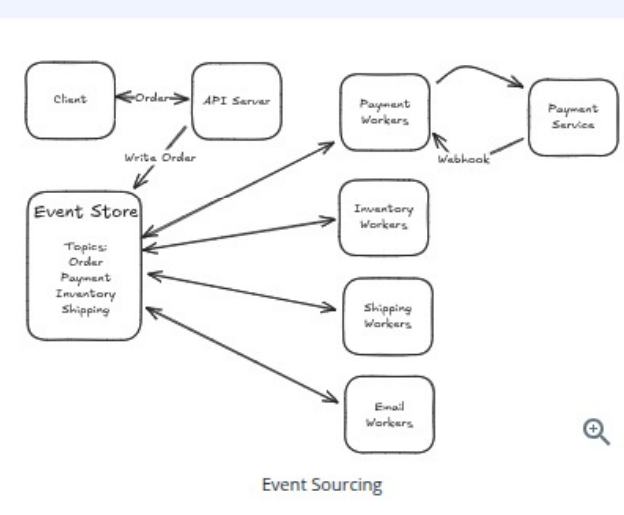
> There's a good chance if you've seen a system like this, it's been an ongoing operational challenge for your company!

Event Sourcing

The most foundational solution to this problem is to use an architectural pattern known as **event sourcing**. Event sourcing offers a more principled approach to our earlier single-server orchestration with state persistence. Instead of storing the current state, you store a sequence of events that represent what happened.

The most common way to store events is to use a durable log and Kafka is a popular choice, although **Redis** Streams could work in some places.

> Event sourcing is a close, but more practical cousin to Event-Driven Architecture. Whereas EDA is about **decoupling** services by publishing events to a topic, event sourcing is about **replaying** events to reconstruct the state of the system with the goal of increasing robustness and reliability.



Event Sourcing

Here's how it works. We're using the logs in event store to store the entire history of the system but also to orchestrate next steps. Whenever something happens that we need to react to, we write an event to the event store and have a worker who can pick it up and react to it. Each worker consumes events, performs its work, and emits new events.

So the payment worker sees "OrderPlaced" and calls our payment service. When the payment service calls back later with the status, the Payment Worker emits "PaymentCharged" or "PaymentFailed". The inventory worker sees "PaymentCharged" and emits "InventoryReserved" or "InventoryFailed". And so on.

Our API service is now just a thin initiating wrapper around the event store. When the order request comes in, we emit an "OrderPlaced" event and the system springs to life to carry the event through the system. Rather than services exposing APIs, they are now just workers who consume events.

> LinkedIn has a great post from 2013 about architecture around durable logs which may help you to understand the intuition behind using durable logs for this pattern.

This gives you:

- **Fault tolerance**: If a worker crashes, another picks up the event
- **Scalability**: Add more workers to handle higher load
- **Observability**: Complete audit trail of all events
- **Flexibility**: Possible to add new steps or modify workflows

Good stuff! But you're building significant infrastructure to make it work like event stores, message queues, and worker orchestration. For complex business processes, this becomes its own distributed systems engineering project.

Also, monitoring and debugging this system can be significantly more complex. Why was there no worker pool to pick up this particular PaymentFailed event? What was the lineage of events that led to this InventoryReserved event? Thousands of redundant internal tools have been built to help with this and good engineers will be skeptical of pat solutions to any of these problems.

Workflows

What we really want to do is to describe a *workflow*, a reliable, long-running processes that can survive failures and continue where they left off. Our ideal system needs to be robust to server crashes or restarts instead of losing all progress and it shouldn't require us to hand-roll the infrastructure to make it work.

Enter workflow systems and durable execution engines. These solutions provide the benefits of event sourcing and state management without requiring you to build the infrastructure yourself. Just like systems like **Flink** provide a way for you to describe streaming event processing at a higher-level, workflow systems and durable execution engines give tools for handling these common multi-step processes. Both provide a language for you to describe the high-level workflow of your system and they handle the orchestration of it, but they differ in how those workflows are described and managed.

Let's describe both briefly:

Durable Execution Engines

Durable execution is a way to write long-running code that can move between machines and survive system failures and restarts. Instead of losing all progress when a server crashes or restarts, durable execution systems automatically resume workflows from their last successful step on a new, running host. Most durable execution engines use *code* to describe the workflow. You write a function that represents the workflow, and the engine handles the orchestration of it.

The most popular durable execution engine is **Temporal**. It's a mature, open-source project (originally built at Uber and called Cadence) that has been around since 2017 and is used by many large companies.

For example, here's a simple workflow in Temporal:

```
const {
    processPayment,
    reserveInventory,
    shipOrder,
    sendConfirmationEmail,
    refundPayment
} = proxyActivities<Activities>({
    startToCloseTimeout: '5 minute',
    retry: {
        maximumAttempts: 3,
    }
});

async function myWorkflow(input: Order): Promise<OrderResult> {
    const paymentResult = await processPayment(input);

    if(paymentResult.success) {
        const inventoryResult = await reserveInventory(input);

        if(inventoryResult.success) {
            await shipOrder(input);
            await sendConfirmationEmail(input);
        } else {
            await refundPayment(input);
            return { success: false, error: "Inventory reservation failed" };
        }
    } else {
        return { success: false, error: "Payment failed" };
    }
}
```

If this looks a lot like the single-server orchestration we saw earlier, that's because it is! The big difference here is how this code is run. Temporal runs workflow code in a special environment that guarantees deterministic execution: timestamps are fixed, random numbers are deterministic, and so on.

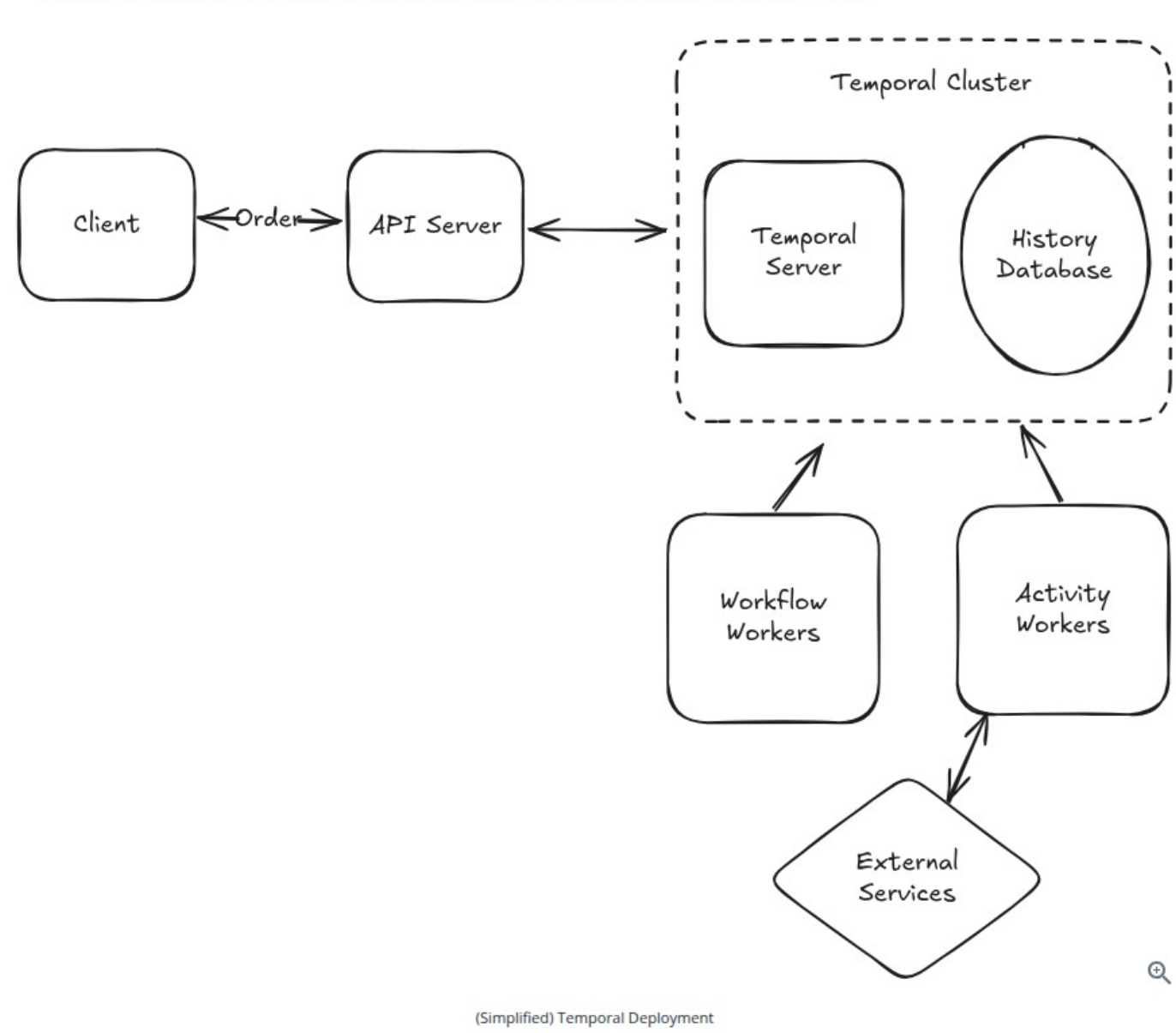
Development of Temporal workflows centers around the concepts of "workflows" and "activities". Workflows are the high-level flow of your system, and activities are the individual steps in that flow. Workflows are *deterministic*: given the same inputs and history, they always make the same decisions. This enables replay-based recovery. Activities need to be *idempotent*: they can be called multiple times with the same inputs and get the same result, but temporal guarantees that they are not retried once they return successfully.

The way this works is each activity run is recorded into a history database. If a workflow runner crashes, another runner can replay the workflow and utilize the history database to remember what happened with each activity invocation: eliminating the need to run the activity again. Activity executions can be spread across many worker machines, and the workflow engine will automatically balance the work across them.

Workflows can also utilize *signals* to wait for external events. For example, if you're waiting for a human to pick up an order, your workflow can wait for a signal that the human has picked up the order before continuing. Most durable execution engines provide a way to wait for signals that is more efficient and lower-latency than polling.

A typical Temporal deployment has a lot in common with our event sourcing architecture earlier:

1. **Temporal Server**: Centralized orchestration that tracks state and manages execution
2. **History Database**: Append-only log of all workflow decisions and activity results
3. **Worker Pools**: Separate pools for workflow orchestration and activity execution



(Simplified) Temporal Deployment

The main differences are that (a) in a Temporal application, the workflow is *explicitly* defined in code vs implicitly defined by the pattern in which workers consume from various topics, and (b) a separate set of workers is needed to execute the user-defined workflow code.

Managed Workflow Systems

Managed workflow systems use a more declarative approach. You define the workflow in a declarative language, and the engine handles the orchestration of it.

The most popular workflow systems include **AWS Step Functions**, **Apache Airflow**, and **Google Cloud Workflows**. Instead of writing code that looks like regular programming, you define workflows as state machines or DAGs (Directed Acyclic Graphs) using JSON, YAML, or specialized DSLs.

Here's the same order fulfillment workflow in AWS Step Functions:

```
{
  "Comment": "Order fulfillment workflow",
  "StartAt": "ProcessPayment",
  "States": {
    "ProcessPayment": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:processPayment",
      "Next": "CheckPaymentResult",
      "Catch": [
        {
          "ErrorEquals": ["States.TaskFailed"],
          "Next": "PaymentFailed"
        }
      ]
    },
    "CheckPaymentResult": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.paymentResult.success",
          "BooleanEquals": true,
          "Next": "ReserveInventory"
        }
      ],
      "Default": "PaymentFailed"
    },
    /** More and more lines of ugly, declarative JSON ... **/
  }
}
```

Ugly but functional!

Under the covers the managed workflow systems are doing the same thing as the durable execution engines: they are orchestrating a workflow, calling activities, and recording progress in such a way that they can be resumed in the case of failures.

The declarative approach to workflow systems brings some advantages. One of the most significant is the ability to visualize workflows as diagrams which means a much nicer UI. This comes with its own set of drawbacks in terms of expressiveness — you may find yourself creating a lot of custom code to fit into the declarative model.

Ultimately, the decision to use a declarative workflow system versus a more code-driven approach depends largely on the preferences of the team and rarely is a point of debate in a system design interview. Both can be made to work for similar purposes.

Implementations

Both approaches provide **durable execution** so your workflow's state persists across failures, restarts, and even code deployments. When a workflow executes an activity, the engine saves a checkpoint. If the server crashes, another worker picks up exactly where it left off. You can write code very similar to the single-server orchestration we saw earlier, but with the added guarantees of fault-tolerance, scalability, and observability.

Temporal is the most powerful open-source option. It provides true durable execution with strong consistency guarantees. Workflows can run indefinitely, survive any failure, and maintain perfect audit trails. The downside is operational complexity - you need to run Temporal clusters in production. Use this when you need maximum control and have the team to operate it.

AWS Step Functions offers serverless workflows if you're already on AWS. You define workflows as state machines in JSON, which is less expressive than code but eliminates operational overhead. It integrates well with other AWS services but has limitations on execution duration (1 year max) and state size (256KB). Choose this for simple orchestration in AWS-heavy environments.

Durable Functions (Azure) and **Google Cloud Workflows** provide similar cloud-native options. They're easier to operate than Temporal but less flexible.

Apache Airflow excels at scheduled batch workflows but wasn't designed for event-driven, long-running processes. It's great for ETL and data pipelines, less suitable for user-facing workflows.

For interviews, default to Temporal unless there's a reason not to. It's the most full-featured and demonstrates you understand the space. Mention Step Functions if the company is AWS-centric and simplicity matters more than power.

When to Use in Interviews

Workflow systems shine in specific scenarios. Don't force them into every design - recognize when their benefits outweigh the complexity.

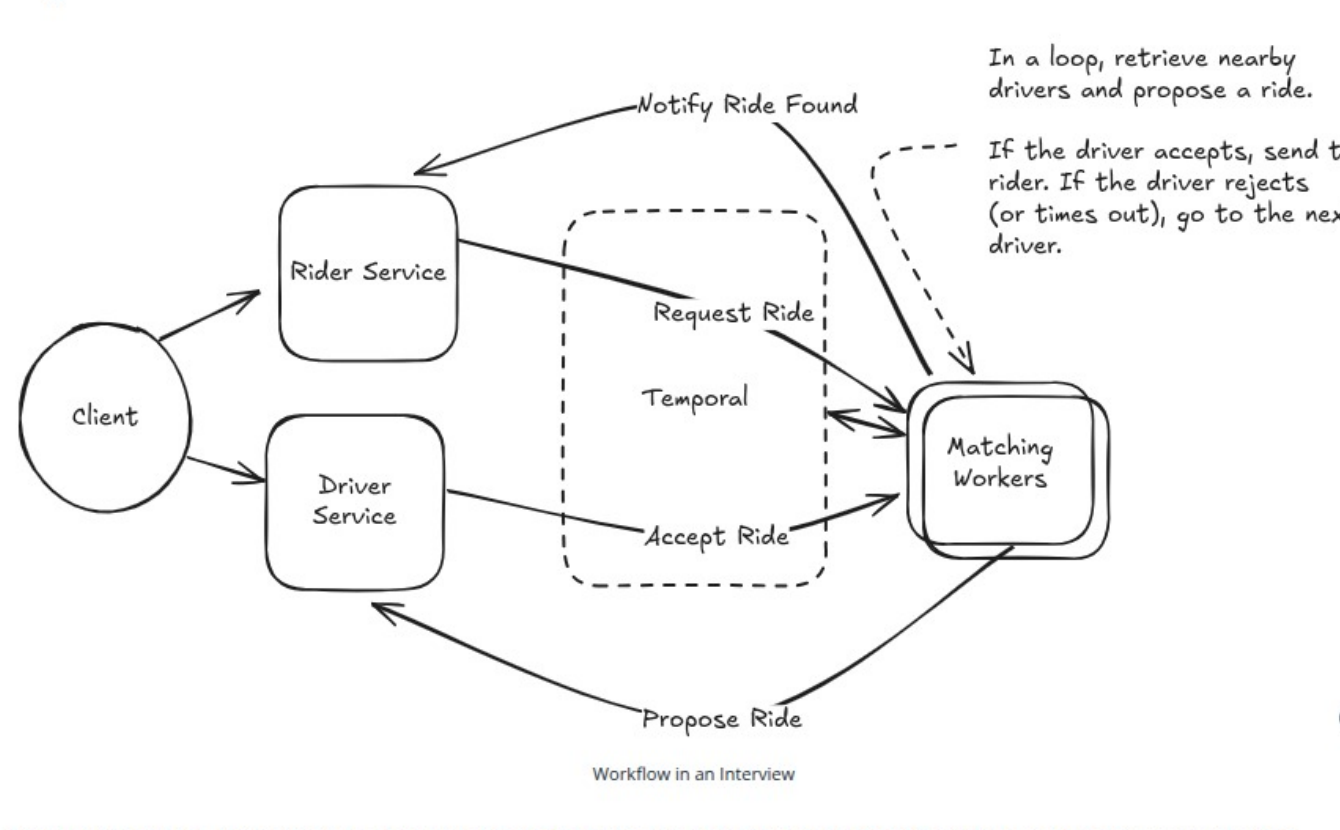
Common interview scenarios

Workflows often show up when there is a *state machine* or a *stateful* process in the design. If you find a sequence of steps that require a flow chart, there's a good chance you should be using a workflow system to design the system around it.

A couple examples:

Payment Systems - In **Payment Systems** or systems that engage with them (like e-commerce systems), there's frequently a lot of state and a strong need to be able to handle failures gracefully. You don't want a user to end up with a charge for a product they didn't receive!

Human-in-the-Loop Workflows - In products like **Uber**, there are a bunch of places where the user is waiting on a human to complete a task. When a user requests a driver, for instance, the driver has to accept the ride. These make for great workflow candidates.


Notify Ride Found
In a loop, retrieve nearby drivers and propose a ride.
If the driver accepts, send to rider. If the driver rejects (or times out), go to the next driver.
Rider Service
Request Ride
Temporal
Client
Driver Service
Matching Workers
Accept Ride
Propose Ride

Workflow in an Interview

In your interview, listen for phrases like "if step X fails, we need to undo step Y" or "we need to ensure all steps complete or none do." That's a clear signal for workflows.

When NOT to use it in an interview

Most CRUD operations, simple request/response APIs, and single-service operations don't need workflows. Don't overcomplicate:

Simple async processing If you just need to resize an image or send an email, use a message queue. Workflows are overkill for single-step operations.

Synchronous operations If the client waits for the response, or there is a lot of sensitivity around latency, you probably don't need (or can't use) a workflow. Save them for truly async, multi-step processes.

High-frequency, low-value operations Workflows add overhead. For millions of simple operations, the cost and complexity aren't justified.

> In interviews, demonstrate maturity by starting simple. Only introduce workflows when you identify specific problems they solve: partial failure handling, long-running processes, complex orchestration, or audit requirements. Show you understand the tradeoffs.

Common Deep Dives

Interviewers often probe specific challenges with workflow systems. Here are the common areas they explore:

"How will you handle updates to the workflow?"

The interviewer asks: "You have 10,000 running workflows for loan approvals. You need to add a new compliance check. How do you update the workflow without breaking existing executions?"

The challenge is that workflows can run for days or weeks. You can't just deploy new code and expect running workflows to handle it correctly. If a workflow started with 5 steps and you deploy a version with 6 steps, what happens when it resumes?

Workflow versioning and workflow migrations are the two main approaches to this.

Workflow Versioning

In workflow versioning, we simply create a new version of the workflow code and deploy it separately. Old workflows will continue to run with the old version of the code, and new workflows will run with the new version of the code.

This is the simplest approach but it's not always feasible. If you need the change to take place immediately, you can't wait for all the legacy workflows to complete.

Workflow Migrations

Workflow migrations are a more complex approach that allows you to update the workflow in place. This is useful if you need to add a new step to the workflow, but you don't want to break existing workflows.

With declarative workflow systems, you can simply update the workflow definition in place. As long as you don't have complex branching or looping logic, both new and existing invocations can follow the new path.

With durable execution engines, you'll often use a "patch" which helps the workflow system to decide *deterministically* whether a new path can be followed. For workflows that have passed through the patched branch before in their execution, they follow the legacy path. For new workflows that have yet to follow the patched branch, they follow the new path.

```
if(workflow.patched("change-behavior")) {
  await a.newBehavior();
} else {
  await a.legacyBehavior();
}
```

"How do we keep the workflow state size in check?"

When using a durable execution engine like Temporal, the entire history of the workflow execution needs to be persisted. When a worker crashes, the workflow is replayed from the beginning, using the results in the history of previous activity invocations instead of re-running the activities. This means your workflow state can grow very large very quickly, and some interviewers like to poke on this.

There's a few aspects of the solution: first, we should try to minimize the size of the activity input and results. If you can pass an identifier which can be looked up in a database or external system rather than a huge payload, you can do that.

Second, we can keep our workflows lean by periodically recreating them. If you have a long-running workflow with lots of history, you can periodically recreate the workflow from the beginning, passing only the required inputs to the new workflow to keep going.

"How do we deal with external events?"

The interviewer asks: "Your workflow needs to wait for a customer to sign documents. They might take 5 minutes or 5 days. How do you handle this efficiently?"

Workflows excel at waiting without consuming resources. Use signals for external events:

```
@workflow
def document_signing_workflow(doc_id):
    # Send document for signing
    yield send_for_signature_activity(doc_id)

    # Wait for signal or timeout
    signed = False
    signature_data = None

    try:
        # Wait up to 30 days for signature
        signature_data = yield wait_for_signal(
            "document_signed",
            timeout=timedelta(days=30)
        )
        signed = True
    except TimeoutError:
        # Handle timeout
        yield send_reminder_activity(doc_id)

        # Wait another 7 days
        try:
            signature_data = yield wait_for_signal(
                "document_signed",
                timeout=timedelta(days=7)
            )
            signed = True
        except TimeoutError:
            yield cancel_document_activity(doc_id)

    if signed:
        yield process_signature_activity(signature_data)
```

External systems send signals through the workflow engine's API. The workflow suspends efficiently - no polling, no resource consumption. This pattern handles human tasks, webhook callbacks, and integration with external systems.

"How can we ensure X step runs exactly once?"

Most workflow systems provide a way to ensure an activity runs *exactly once* ... for a very specific definition of "run". If the activity finishes successfully, but fails to "ack" to the workflow engine, the workflow engine will retry the activity. This might be a bad thing if the activity is a refund or an email send.

The solution is to make the activity **idempotent**. This means that the activity can be called multiple times with the same inputs and get the same result. Storing off a key to a database (e.g. the idempotency key of the email) and then checking if it exists before performing the irreversible action is a common pattern to accomplish this.

Conclusion

Workflow systems are a perfect fit for hairy state machines that are otherwise difficult to get right. They allow you to build reliable distributed systems by centralizing state management, retry logic, and error handling in a purpose-built engine. This lets us write business logic that reads like business requirements, not infrastructure gymnastics.

The key insight is recognizing when you're manually building what a workflow engine provides: state persistence across failures, orchestration of multiple services, handling of long-running processes, and automatic retries with compensation. If you find yourself implementing distributed sagas by hand or building state machines in Redis, it's time to consider a workflow system.

Be ready to discuss the tradeoffs. Yes, you're adding operational complexity with another distributed system to manage. Yes, there's a learning curve for developers. But for the right problems workflow systems transform fragile manual orchestration into robust, observable, and maintainable solutions.

Test Your Knowledge
Take a quick 15 question quiz to test what you've learned.
Start Quiz

Mark as read

Add a comment...

Comment

Anonymous
Posting as leetcode

Search comments
Sort By
Old

Hui Qi • 1 month ago • edited 1 month ago
cool article, but i am confused when we should mention this in the interview even with the `When to Use in Interviews` section. e.g. I couldn't find it in the Design Uber, and what level it is targeting? deep dive for senior or staff? Thanks!
0 Reply

Stefan Mai • 1 month ago
Deep Dive 5. Could show up at either level.
5 Reply

RainyCyanHippopotamus256 • 1 month ago • edited 1 month ago
Could you please elaborate a bit more on why do we generally use Airflow for ETLs and AWS StepFunctions for real-time or event-driven workflows? How do their designs differ due to which we prefer one over another for different use-cases?
1 Reply

Stefan Mai • 1 month ago
Airflow tasks are typically long-running batch operations that process large amounts of data, while Step Functions executes shorter, more interactive steps (like API calls or Lambda functions). Also worth noting that Airflow's DAG structure is more rigid - once a DAG starts executing, you can't dynamically change the flow based on results. Step Functions supports dynamic branching and complex state machines, which you often need in event-driven workflows.
4 Reply

Shiksha Sharma • 9 days ago
in event drivem like Payment system dont we use kafka to store events why is Step needed for event driven - step is AWS's so how does it fit into bigger picture
0 Reply

VitreousAmberEmu523 • 1 month ago • edited 1 month ago
For the managed services solution from AWS, article mentions the StepFunctions. However, I want to point out that it's just the serverless offering and more powerful and comparable option to temporal is actually AWS SimpleWorkfow Service. Probably article should include SWF service instead.

Also an interesting fact, Creators of temporal are the creators of the AWS SWF service. These distinguished engineers created SWF while at AWS and then moved to uber and developed cadence and what's temporal today.
7 Reply

prasad parshallu • 1 month ago • edited 1 month ago
First off, thank you for sharing these patterns—they're incredibly helpful for reasoning through one system versus another. I do have a question about idempotency. Suppose we store an idempotency key and status in the database for tracking instead of ttl, but the service crashes before the business logic executes. In that case, we can safely retry the logic based on the status. However, what if the service crashes *after* the business logic runs but before the status is updated? How to prevent redundant or duplicate executions in this scenario?
0 Reply

Stefan Mai • 1 month ago
Yeah good question, you're butting up against a fundamental limitation here. Note this applies universally: the scope of "exactly once X" or "at least once X" semantics depends a bit on where we draw the line around X, 2 phase commit can fail at the commit stage, etc etc. As long as X and recording execution of X are not atomic together, there's always a chance of failure and generally our job is to design in such a way that this failure mode is minimized.

If this was a really important activity `administerMedication`, what you would do is to record status to a idempotency key on common storage when it starts and when it finishes. So set `IN_PROGRESS` and then `COMPLETED`. You can now guarantee 3 states:

1. Status is NOT_STARTED. The medicine hasn't been administered, proceed.
2. Status is IN_PROGRESS. The medicine *may* have been administered.
3. Status is COMPLETED. The medicine definitely has been administered.

We need some fallback for that 2nd state. One solution is to record the result of the administration *to local disk* before we go to update remote status with our idempotency key. Then we can enforce a process wherein we go recover the disk after the server crash in cases where our status reports IN_PROGRESS for longer than expected. Now we've limited our surface area to just that disk flush.

1. Status is NOT_STARTED. The medicine hasn't been administered, proceed.

Show More
8 Reply

Shiksha Sharma • 9 days ago
local disk ? I didnt get it
is it consumer local disk? Will other consumer have access to this local disk?
and also is it used somewhere?
0 Reply

Shiksha Sharma • 9 days ago
@Stefan Mai - for the question prasad parshallu has asked, in long running write up haven't we asnwered this by saying that make your worker as idempotent?
0 Reply

Yves Sy • 3 days ago • edited 3 days ago
Just ask the patient if you fall under #3 😂
1 Reply

Stefan Mai • 3 days ago • edited 3 days ago
🤣 anything to avoid user interaction
0 Reply

Jose • 1 month ago • edited 1 month ago
Great article—thank you!
You list Kafka/Redis as event-store options, but I keep running into two issues:

- Reads: replaying a single stream (e.g., order-123) means scanning an entire partition or maintaining an external projection.
- Writes: if we buffer/project outside Kafka, the next event may not be appended directly after the previous one, so ordering and atomicity weakens. What pattern do you use to satisfy both while still treating Kafka as the event store? (some notes may resemble my thoughts)

0 Reply

Show All Comments

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

Schedule A Mock Interview

Questions
Meta SWE Interview Questions
Amazon SWE Interview Questions
Google SWE Interview Questions
Engineering Manager (EM) Interview Questions

Learn
Learn System Design
Learn DSA
Learn Behavioral
Learn ML System Design
Guided Practice

Links
FAQ
Pricing
Gift Mock Interviews
Gift Premium
Become a Coach
Our Coaches
Employers
Hello Interview Premium

Legal
Terms and Conditions
Privacy Policy

Contact
About Us
support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.